

LAPORAN TUGAS PEKAN 9

STRUKTUR DATA



Disusun oleh:

FAHRI SYAH PUTRA RAMADHAN

2511531015

Dosen pengampu:

Dr. Wahyudi. MT

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG, 2026

KATA PENGANTAR

Puji syukur penulis ucapkan ke hadirat Allah SWT karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum ini dengan baik. Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah Struktur Data.

Laporan ini membahas mengenai implementasi struktur data **Binary Tree** dan **Graph Traversal** menggunakan bahasa pemrograman Java. Pada bagian Binary Tree, program dibuat untuk membentuk pohon biner, menambahkan simpul, menghitung jumlah simpul, serta menampilkan data menggunakan metode traversal seperti InOrder, PreOrder, dan PostOrder. Sementara itu, pada bagian Graph Traversal, program dibuat untuk merepresentasikan graf menggunakan adjacency list serta melakukan penelusuran menggunakan metode DFS dan BFS.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi pembahasan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik ke depannya.

Akhir kata, penulis berharap laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami konsep Binary Tree, Graph Traversal, DFS, BFS, dan penerapannya dalam bahasa pemrograman Java.

Padang, 2026

FAHRI SYAH PUTRA RAMADAHAN

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
BAB II PEMBAHASAN	2
2.1 Penjelasan Kode Program Node_2511531015	2
2.2 Penjelasan Kode Program BTree_2511531015	5
2.3 Penjelasan Kode Program BtreeDriver_2511531015	8
2.4 Penjelasan Kode Program GraphTraversal_2511531015	11
2.5 Foto Kode Program dan Output	15
BAB III PENUTUP	18
3.1 Kesimpulan	18

PENJELASAN KODE PROGRAM

NODE_2511531015

Program ini adalah class **Node_2511531015** yang digunakan untuk membuat **simpul/node** pada **struktur data Binary Tree**.

Binary Tree adalah struktur data berbentuk pohon, di mana setiap node maksimal memiliki **dua anak**, yaitu:

Anak kiri -> left

Anak kanan -> right

Jadi, class ini berfungsi untuk menyimpan data, menghubungkan node ke kiri dan kanan, serta mencetak isi pohon dalam beberapa urutan traversal.

1. Package

```
package pekan9_2511531015;
```

Kode ini menunjukkan bahwa class `Node_2511531015` berada di dalam package bernama `pekan9_2511531015`.

Artinya, file ini harus berada di folder/package yang sama dengan class lain seperti:

```
BTree_2511531015
```

```
BtreeDriver_2511531015
```

2. Deklarasi Class

```
public class Node_2511531015 {
```

Bagian ini mendeklarasikan class bernama `Node_2511531015`.

Class ini digunakan sebagai cetakan untuk membuat objek node.

Contoh:

```
Node_2511531015 root_1015 = new Node_2511531015(1);
```

Artinya membuat node baru dengan data bernilai 1.

3. Atribut pada Node

```
int data_1015;
```

```
Node_2511531015 left_1015;
```

```
Node_2511531015 right_1015;
```

Bagian ini adalah atribut atau variabel yang dimiliki oleh setiap node.

data_1015 digunakan untuk menyimpan nilai data pada node.

```
int data_1015;
```

Contohnya node bisa menyimpan angka:

1, 2, 3, 4, 5

left_1015 digunakan untuk menyimpan alamat anak kiri.

```
Node_2511531015 left_1015;
```

right_1015 digunakan untuk menyimpan alamat anak kanan.

```
Node_2511531015 right_1015;
```

Secara sederhana, bentuk satu node adalah seperti ini:

```
    data
  /   \
left   right
```

4. Constructor

```
public Node_2511531015(int data_1015) {
    this.data_1015 = data_1015;
    left_1015 = null;
    right_1015 = null;
}
```

Constructor adalah method khusus yang dijalankan saat objek dibuat.

Contoh:

```
Node_2511531015 node1 = new Node_2511531015(1);
```

Maka nilai data_1015 akan diisi dengan angka 1.

Bagian ini:

```
this.data_1015 = data_1015;
```

berarti nilai dari parameter data_1015 dimasukkan ke variabel milik class.

Sedangkan:

```
left_1015 = null;
```

```
right_1015 = null;
```

berarti saat node baru dibuat, node tersebut belum memiliki anak kiri dan anak kanan.

5. Method setleft_1015

```
public void setleft_1015(Node_2511531015 node) {  
    if (left_1015 == null)  
        left_1015 = node;  
}
```

Method ini digunakan untuk mengisi anak kiri dari sebuah node.

Contoh:

```
root_1015.setleft_1015(node2_1015);
```

Artinya node2_1015 dijadikan anak kiri dari root_1015.

Bagian:

```
if (left_1015 == null)
```

berarti anak kiri hanya akan diisi jika masih kosong.

Jadi, jika anak kiri sudah ada, maka tidak akan diganti.

6. Method setright_1015

```
public void setright_1015(Node_2511531015 node) {  
    if (right_1015 == null)  
        right_1015 = node;  
}
```

Method ini digunakan untuk mengisi anak kanan dari sebuah node.

Contoh:

```
root_1015.setright_1015(node3_1015);
```

Artinya node3_1015 dijadikan anak kanan dari root_1015.

Sama seperti anak kiri, anak kanan hanya akan diisi jika masih kosong.

7. Method Getter Anak Kiri dan Anak Kanan

```
public Node_2511531015 getleft_1015() {  
    return left_1015;  
}
```

Method ini digunakan untuk mengambil anak kiri dari sebuah node.

Sedangkan:

```
public Node_2511531015 getright_1015() {  
    return right_1015;  
}
```

digunakan untuk mengambil anak kanan.

Contoh:

```
root_1015.getleft_1015();  
root_1015.getright_1015();
```

8. Method Getter dan Setter Data

```
public int getdata_1015() {  
    return data_1015;  
}
```

Method ini digunakan untuk mengambil nilai data pada node.

Contoh:

```
root_1015.getdata_1015();
```

Jika data root bernilai 1, maka hasilnya adalah:

1

Kemudian:

```
public void setdata_1015(int data_1015) {  
    this.data_1015 = data_1015;  
}
```

Method ini digunakan untuk mengubah nilai data pada node.

Contoh:

```
root_1015.setdata_1015(10);
```

Artinya data pada node root diubah menjadi 10.

9. Method printPreorder

```
void printPreorder(Node_2511531015 node) {  
    if (node == null)  
        return;  
    System.out.print(node.data_1015 + " ");  
    printPreorder(node.left_1015);  
    printPreorder(node.right_1015);  
}
```

Method ini digunakan untuk mencetak isi binary tree dengan urutan **Preorder**.

Urutan Preorder adalah:

Root - Left - Right

Artinya:

1. Cetak node utama terlebih dahulu.
2. Lanjut ke anak kiri.
3. Lanjut ke anak kanan.

Contoh jika bentuk pohonnya seperti ini:

```
  1  
 /\  
2 3
```

Maka hasil Preorder adalah:

1 2 3

10. Method **printPostorder**

```
void printPostorder(Node_2511531015 node) {  
    if (node == null)  
        return;  
    printPostorder(node.left_1015);  
    printPostorder(node.right_1015);  
    System.out.print(node.data_1015 + " ");  
}
```

Method ini digunakan untuk mencetak isi binary tree dengan urutan **Postorder**.

Urutan Postorder adalah:

Left - Right - Root

Artinya:

1. Cetak anak kiri terlebih dahulu.
2. Cetak anak kanan.
3. Cetak node utama terakhir.

Contoh:

```
    1  
   /\n  2 3
```

Maka hasil Postorder adalah:

2 3 1

11. Method printInorder

```
void printInorder(Node_2511531015 node) {  
    if (node == null)  
        return;  
    printInorder(node.left_1015);  
    System.out.print(node.data_1015 + " ");  
    printInorder(node.right_1015);  
}
```

Method ini digunakan untuk mencetak isi binary tree dengan urutan **Inorder**.

Urutan Inorder adalah:

Left - Root - Right

Artinya:

1. Cetak anak kiri.
2. Cetak node utama.
3. Cetak anak kanan.

Contoh:

```
  1  
 /\  
2 3
```

Maka hasil Inorder adalah:

2 1 3

12. Method print

```
public String print() {  
    return this.print("", true, "");  
}
```

Method ini digunakan untuk menampilkan bentuk tree secara visual.

Method ini memanggil method lain, yaitu:

```
print(String prefix, boolean isTail, String sb)
```

Tujuannya agar pohon bisa ditampilkan seperti struktur bercabang.

13. Method Cetak Bentuk Pohon

```
public String print(String prefix, boolean isTail, String sb) {  
    if (right_1015 != null) {  
        right_1015.print(prefix + (isTail ? "| " : " "), false, sb);  
    }  
    System.out.println(prefix + (isTail ? "\\-- " : "/-- ") + data_1015);  
    if (left_1015 != null) {  
        left_1015.print(prefix + (isTail ? " " : "| "), true, sb);  
    }  
    return sb;  
}
```

Method ini digunakan untuk mencetak bentuk pohon secara miring di layar.

Bagian ini mencetak anak kanan terlebih dahulu:

```
if (right_1015 != null) {  
    right_1015.print(prefix + (isTail ? "| " : " "), false, sb);  
}
```

Kemudian mencetak node saat ini:

```
System.out.println(prefix + (isTail ? "\\-- " : "/-- ") + data_1015);
```

Lalu mencetak anak kiri:

```
if (left_1015 != null) {  
    left_1015.print(prefix + (isTail ? "    " : "|  "), true, sb);  
}
```

Artinya node 1 adalah root, node 2 berada di kiri, dan node 3 berada di kanan.

```
| /-- 3
```

```
\-- 1
```

```
    \-- 2
```

BTREE_2511531015

Class **BTree_2511531015** digunakan untuk mengelola struktur data **Binary Tree** atau pohon biner.

Binary Tree adalah struktur data berbentuk pohon, di mana setiap simpul atau node dapat memiliki maksimal dua anak, yaitu:

Anak kiri -> left

Anak kanan -> right

Class ini berfungsi untuk menyimpan root pohon, mencari data, menghitung jumlah node, mencetak isi pohon dengan beberapa metode traversal, serta menentukan node yang sedang aktif atau currentNode.

1. Package

```
package pekan9_2511531015;
```

Kode ini menunjukkan bahwa class **BTree_2511531015** berada di dalam package bernama:

```
pekan9_2511531015
```

Jadi file ini harus berada dalam folder/package yang sama dengan file lain seperti:

```
Node_2511531015.java
```

```
BtreeDriver_2511531015.java
```

2. Deklarasi Class

```
public class BTree_2511531015 {
```

Bagian ini mendeklarasikan class bernama **BTree_2511531015**.

Class ini digunakan untuk membentuk dan mengelola pohon biner.

3. Atribut Root dan Current Node

```
private Node_2511531015 root_1015;
```

```
private Node_2511531015 currentNode;
```

Terdapat dua atribut utama.

Pertama:

```
private Node_2511531015 root_1015;
```

root_1015 digunakan untuk menyimpan node utama atau akar dari pohon.

Root adalah node paling atas dalam struktur tree.

Contohnya:

```
1
/\
2 3
```

Pada contoh tersebut, node 1 adalah root.

Kedua:

```
private Node_2511531015 currentNode;
```

currentNode digunakan untuk menyimpan node yang sedang aktif atau sedang ditunjuk.

Biasanya digunakan ketika program ingin menampilkan data dari node tertentu.

4. Constructor

```
public BTree_2511531015() {
    root_1015 = null;
}
```

Constructor ini dijalankan ketika objek BTree_2511531015 dibuat.

Contoh:

```
BTree_2511531015 tree_1015 = new BTree_2511531015();
```

Saat pertama kali dibuat, nilai root_1015 masih null.

Artinya pohon masih kosong dan belum memiliki node.

5. Method search

```
public boolean search(int data_1015) {
    return search(root_1015, data_1015);
}
```

Method ini digunakan untuk mencari apakah sebuah data ada di dalam pohon.

Contoh:

```
tree_1015.search(7);
```

Artinya program akan mencari apakah angka 7 ada di dalam binary tree.

Method ini memanggil method search yang bersifat private:

```
return search(root_1015, data_1015);
```

Artinya pencarian dimulai dari root.

6. Method search Private

```
private boolean search(Node_2511531015 node_1015, int data_1015) {  
    if (node_1015.getdata_1015() == data_1015)  
        return true;  
    if (node_1015.getleft_1015() != null)  
        if (search(node_1015.getleft_1015(), data_1015))  
            return true;  
    if (node_1015.getright_1015() != null)  
        if (search(node_1015.getright_1015(), data_1015))  
            return true;  
    return false;  
}
```

Method ini adalah proses pencarian sebenarnya.

Cara kerjanya:

1. Program mengecek apakah data pada node saat ini sama dengan data yang dicari.
2. Jika sama, maka mengembalikan nilai true.
3. Jika tidak sama, program mencari ke anak kiri.
4. Jika belum ditemukan, program mencari ke anak kanan.
5. Jika tetap tidak ditemukan, maka mengembalikan nilai false.

Bagian ini:

```
if (node_1015.getdata_1015() == data_1015)  
    return true;
```

berarti jika data pada node sama dengan data yang dicari, maka data ditemukan.

Bagian ini:

```
if (node_1015.getleft_1015() != null)
```

berarti program akan mencari ke anak kiri jika anak kiri tidak kosong.

Bagian ini:

```
if (node_1015.getright_1015() != null)
```

berarti program akan mencari ke anak kanan jika anak kanan tidak kosong.

Catatan penting: method ini bisa error jika pohon masih kosong, karena belum ada pengecekan:

```
if (node_1015 == null)
```

Versi yang lebih aman sebaiknya seperti ini:

```
private boolean search(Node_2511531015 node_1015, int data_1015) {
```

```
    if (node_1015 == null)
```

```
        return false;
```

```
    if (node_1015.getdata_1015() == data_1015)
```

```
        return true;
```

```
    if (search(node_1015.getleft_1015(), data_1015))
```

```
        return true;
```

```
    if (search(node_1015.getright_1015(), data_1015))
```

```
        return true;
```

```
    return false;
```

```
}
```

7. Method printInorder

```
public void printInorder() {  
    root_1015.printInorder(root_1015);  
}
```

Method ini digunakan untuk mencetak isi pohon dengan urutan **Inorder**.

Urutan Inorder adalah:

Left - Root - Right

Artinya:

1. Cetak anak kiri.
2. Cetak node utama.
3. Cetak anak kanan.

Contoh jika pohonnya:

```
  1  
 /\  
2 3
```

Maka hasil Inorder adalah:

2 1 3

8. Method printPreOrder

```
public void printPreOrder() {  
    root_1015.printPreorder(root_1015);  
}
```

Method ini digunakan untuk mencetak isi pohon dengan urutan **Preorder**.

Urutan Preorder adalah:

Root - Left - Right

Artinya:

1. Cetak node utama.
2. Cetak anak kiri.
3. Cetak anak kanan.

Contoh:

```
1
 / \
2  3
```

Maka hasil Preorder adalah:

1 2 3

9. Method printPostOrder

```
public void printPostOrder() {
    root_1015.printPostorder(root_1015);
}
```

Method ini digunakan untuk mencetak isi pohon dengan urutan **Postorder**.

Urutan Postorder adalah:

Left - Right - Root

Artinya:

1. Cetak anak kiri.
2. Cetak anak kanan.
3. Cetak node utama.

Contoh:

```
1
 / \
2  3
```

Maka hasil Postorder adalah:

2 3 1

10. Method getRoot

```
public Node_2511531015 getRoot() {  
    return root_1015;  
}
```

Method ini digunakan untuk mengambil node root dari pohon.

Contoh penggunaan:

```
tree_1015.getRoot();
```

Method ini akan mengembalikan root dari tree.

Biasanya digunakan saat ingin mengatur current node:

```
tree_1015.setCurrent(tree_1015.getRoot());
```

11. Method isEmpty

```
public boolean isEmpty() {  
    return root_1015 == null;  
}
```

Method ini digunakan untuk mengecek apakah pohon masih kosong atau tidak.

Jika `root_1015 == null`, artinya pohon belum memiliki node.

Contoh:

```
tree_1015.isEmpty();
```

Jika pohon kosong, hasilnya:

true

Jika pohon sudah memiliki root, hasilnya:

false

12. Method countNodes

```
public int countNodes() {  
    return countNodes(root_1015);  
}
```

Method ini digunakan untuk menghitung jumlah node yang ada di dalam pohon.

Method ini memanggil method private:

```
countNodes(root_1015);
```

Artinya proses penghitungan dimulai dari root.

13. Method countNodes Private

```
private int countNodes(Node_2511531015 node_1015) {  
    int count_1015 = 1;  
    if (node_1015 == null) {  
        return 0;  
    } else {  
        count_1015 += countNodes(node_1015.getleft_1015());  
        count_1015 += countNodes(node_1015.getright_1015());  
        return count_1015;  
    }  
}
```

Method ini digunakan untuk menghitung jumlah seluruh node secara rekursif.

Bagian ini:

```
int count_1015 = 1;
```

berarti setiap node yang tidak kosong dihitung sebagai 1.

Bagian ini:

```
if (node_1015 == null) {  
    return 0;  
}
```

berarti jika node kosong, maka jumlahnya 0.

Bagian ini:

```
count_1015 += countNodes(node_1015.getleft_1015());  
count_1015 += countNodes(node_1015.getright_1015());
```

berarti program menghitung jumlah node di cabang kiri dan cabang kanan.

Contoh pohon:

```
1
 /\
2 3
```

Jumlah node-nya adalah:

3

Karena ada node 1, 2, dan 3.

14. Method print

```
public void print() {
    root_1015.print();
}
```

Method ini digunakan untuk menampilkan bentuk pohon secara visual.

Method ini memanggil method print() yang ada di class Node_2511531015.

Contoh outputnya bisa berbentuk seperti ini:

```
| /-- 3
|-- 1
   |-- 2
```

Artinya node 1 adalah root, node 2 berada di kiri, dan node 3 berada di kanan.

15. Method getCurrent

```
public Node_2511531015 getCurrent() {
    return currentNode;
}
```

Method ini digunakan untuk mengambil node yang sedang aktif atau sedang ditunjuk.

Contoh:

```
tree_1015.getCurrent();
```

Biasanya digunakan bersama:

```
tree_1015.getCurrent().getdata_1015();
```

Kode tersebut berarti mengambil data dari current node.

16. Method setCurrent

```
public void setCurrent(Node_2511531015 node_1015) {  
    this.currentNode = node_1015;  
}
```

Method ini digunakan untuk mengatur node yang sedang aktif.

Contoh:

```
tree_1015.setCurrent(tree_1015.getRoot());
```

Artinya current node diatur menjadi root.

Jika root berisi data 1, maka current node sekarang menunjuk ke node 1.

17. Method setRoot

```
public void setRoot(Node_2511531015 root_1015) {  
    this.root_1015 = root_1015;  
}
```

Method ini digunakan untuk mengatur root atau akar pohon.

Contoh:

```
Node_2511531015 root_1015 = new Node_2511531015(1);  
tree_1015.setRoot(root_1015);
```

Artinya node bernilai 1 dijadikan sebagai root dari binary tree.

BTREEDRIVER_2511531015

Class **BtreeDriver_2511531015** berfungsi sebagai class utama untuk menjalankan program Binary Tree. Di dalam class ini terdapat method `main()` yang digunakan untuk membuat objek pohon, membuat beberapa node, menghubungkan node menjadi struktur pohon, lalu menampilkan hasil traversal dan bentuk pohonnya.

Program ini bekerja sama dengan dua class lain, yaitu:

Node_2511531015

BTree_2511531015

Class Node_2511531015 digunakan untuk membuat simpul/node, sedangkan class BTree_2511531015 digunakan untuk mengelola pohon secara keseluruhan.

1. Package

```
package pekan9_2511531015;
```

Bagian ini menunjukkan bahwa file BtreeDriver_2511531015.java berada di dalam package bernama:

```
pekan9_2511531015
```

Artinya, file ini harus berada satu package dengan file:

```
Node_2511531015.java
```

```
BTree_2511531015.java
```

2. Deklarasi Class

```
public class BtreeDriver_2511531015 {
```

Bagian ini mendeklarasikan class bernama **BtreeDriver_2511531015**.

Class ini berperan sebagai class driver, yaitu class yang digunakan untuk menjalankan program.

3. Method main

```
public static void main(String[] args) {
```

Method `main()` adalah method utama yang pertama kali dijalankan oleh Java.

Semua proses pembuatan binary tree dilakukan di dalam method ini.

4. Membuat Objek Binary Tree

```
BTree_2511531015 tree_1015 = new BTree_2511531015();
```

Kode ini digunakan untuk membuat objek binary tree baru dengan nama:

```
tree_1015
```

Pada awalnya, pohon masih kosong karena belum memiliki root.

5. Menampilkan Jumlah Simpul Awal

```
System.out.print("Jumlah Simpul awal pohon: ");
```

```
System.out.println(tree_1015.countNodes());
```

Kode ini digunakan untuk menampilkan jumlah simpul awal pada pohon.

Karena pohon masih kosong, maka hasilnya adalah:

```
0
```

Method countNodes() digunakan untuk menghitung jumlah node yang ada di dalam pohon.

6. Membuat Node Root

```
Node_2511531015 root_1015 = new Node_2511531015(1);
```

Kode ini digunakan untuk membuat node baru dengan data bernilai 1.

Node ini nantinya akan dijadikan sebagai root atau akar pohon.

Secara sederhana:

```
1
```

7. Menjadikan Node 1 sebagai Root

```
tree_1015.setRoot(root_1015);
```

Kode ini digunakan untuk menjadikan root_1015 sebagai root dari binary tree.

Setelah kode ini dijalankan, pohon sudah memiliki satu simpul, yaitu node 1.

8. Menampilkan Jumlah Simpul Setelah Root Dibuat

```
System.out.println("Jumlah simpul jika hanya ada root");  
System.out.println(tree_1015.countNodes());
```

Kode ini digunakan untuk menghitung jumlah simpul setelah root dimasukkan.

Karena hanya ada satu node, yaitu node 1, maka hasilnya adalah:

1

9. Membuat Node 2 Sampai Node 9

```
Node_2511531015 node2_1015 = new Node_2511531015(2);  
Node_2511531015 node3_1015 = new Node_2511531015(3);  
Node_2511531015 node4_1015 = new Node_2511531015(4);  
Node_2511531015 node5_1015 = new Node_2511531015(5);  
Node_2511531015 node6_1015 = new Node_2511531015(6);  
Node_2511531015 node7_1015 = new Node_2511531015(7);  
Node_2511531015 node8_1015 = new Node_2511531015(8);  
Node_2511531015 node9_1015 = new Node_2511531015(9);
```

Bagian ini digunakan untuk membuat node baru dengan nilai data dari 2 sampai 9.

Setiap node menyimpan satu nilai angka.

10. Menghubungkan Node ke Dalam Tree

```
root_1015.setleft_1015(node2_1015);
```

Kode ini menjadikan node 2 sebagai anak kiri dari root 1.

Strukturnya menjadi:

```
  1  
 /  
2
```

Kemudian:

```
node2_1015.setleft_1015(node4_1015);  
node2_1015.setright_1015(node5_1015);
```

Kode ini menjadikan node 4 sebagai anak kiri dari node 2, dan node 5 sebagai anak kanan dari node 2.

Strukturnya menjadi:

```
1
/
2
/\
4 5
```

Lalu:

```
node4_1015.setright_1015(node8_1015);
```

Kode ini menjadikan node 8 sebagai anak kanan dari node 4.

Kemudian:

```
root_1015.setright_1015(node3_1015);
```

Kode ini menjadikan node 3 sebagai anak kanan dari root 1.

Strukturnya menjadi:

```
1
/\
2 3
/\
4 5
\
8
```

Selanjutnya:

```
node3_1015.setleft_1015(node6_1015);
node3_1015.setright_1015(node7_1015);
```

Kode ini menjadikan node 6 sebagai anak kiri dari node 3, dan node 7 sebagai anak kanan dari node 3.

Terakhir:

```
node6_1015.setleft_1015(node9_1015);
```

Kode ini menjadikan node 9 sebagai anak kiri dari node 6.

Jadi bentuk akhir pohonnya adalah:

```
1
 / \
2   3
 / \ / \
4 5 6 7
 \ /
 8 9
```

11. Mengatur Current Node

```
tree_1015.setCurrent(tree_1015.getRoot());
```

Kode ini digunakan untuk mengatur currentNode agar menunjuk ke root.

Karena root berisi data 1, maka current node saat ini adalah node 1.

12. Menampilkan Data Current Node

```
System.out.println("menampilkan simpul terakhir: ");
```

```
System.out.println(tree_1015.getCurrent().getdata_1015());
```

Kode ini digunakan untuk menampilkan data dari current node.

Karena current node diset ke root, maka yang ditampilkan adalah:

```
1
```

Catatan: teks "menampilkan simpul terakhir" kurang tepat, karena sebenarnya yang ditampilkan adalah **simpul current**, bukan simpul terakhir.

13. Menghitung Jumlah Semua Simpul

```
System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
```

```
System.out.println(tree_1015.countNodes());
```

Kode ini digunakan untuk menghitung jumlah seluruh node yang ada di dalam pohon.

Karena sudah dibuat node dari 1 sampai 9, maka jumlah simpulnya adalah:

```
9
```

Catatan: kalimat "setelah simpul 7 ditambahkan" kurang tepat karena setelah itu masih ada node 8 dan 9. Lebih baik ditulis:

```
System.out.println("Jumlah simpul setelah semua simpul ditambahkan");
```

14. Menampilkan Traversal InOrder

```
System.out.println("InOrder: ");
```

```
tree_1015.printInorder();
```

Kode ini digunakan untuk mencetak isi pohon dengan urutan **InOrder**.

Urutan InOrder adalah:

Left - Root - Right

Artinya program mencetak anak kiri terlebih dahulu, lalu root, kemudian anak kanan.

15. Menampilkan Traversal PreOrder

```
System.out.println("\nPreorder: ");
```

```
tree_1015.printPreOrder();
```

Kode ini digunakan untuk mencetak isi pohon dengan urutan **PreOrder**.

Urutan PreOrder adalah:

Root - Left - Right

Artinya program mencetak root terlebih dahulu, lalu anak kiri, kemudian anak kanan.

16. Menampilkan Traversal PostOrder

```
System.out.println("\nPostorder : ");
```

```
tree_1015.printPostOrder();
```

Kode ini digunakan untuk mencetak isi pohon dengan urutan **PostOrder**.

Urutan PostOrder adalah:

Left - Right - Root

Artinya program mencetak anak kiri terlebih dahulu, lalu anak kanan, kemudian root dicetak terakhir.

17. Menampilkan Bentuk Pohon

```
System.out.println("\nDmenampilkan simpul dalam bentuk pohon");
```

```
tree_1015.print();
```

Kode ini digunakan untuk menampilkan struktur binary tree dalam bentuk visual.

Namun, ada sedikit kesalahan penulisan pada kalimat:

```
"Dmenampilkan simpul dalam bentuk pohon"
```

Seharusnya:

```
"Menampilkan simpul dalam bentuk pohon"
```

Kode yang lebih rapi:

```
System.out.println("\nMenampilkan      simpul      dalam      bentuk      pohon");  
tree_1015.print();
```

GRAPH TRAVERSAL_2511531015

Program ini digunakan untuk membuat dan menelusuri **graf tidak berarah** menggunakan dua metode, yaitu:

DFS = Depth First Search

BFS = Breadth First Search

Graf adalah struktur data yang terdiri dari **node/simpul** dan **edge/garis penghubung**. Pada program ini, graf dibuat menggunakan **Adjacency List**, yaitu setiap node menyimpan daftar node lain yang terhubung dengannya.

Contohnya:

A -> B, C

B -> A, D, E

C -> A

Artinya node A terhubung dengan B dan C.

1. Package

```
package pekan9_2511531015;
```

Kode ini menunjukkan bahwa class berada di dalam package bernama:

```
pekan9_2511531015
```

Jadi file Java ini harus berada di dalam folder/package yang sama.

2. Import Library

```
import java.util.*;
```

Kode ini digunakan untuk mengimpor semua class dari package java.util.

Library ini dibutuhkan karena program menggunakan beberapa struktur data seperti:

Map

List

HashMap

ArrayList

Set

HashSet

Queue

LinkedList

Struktur data tersebut dipakai untuk menyimpan graf, menyimpan node yang sudah dikunjungi, dan membuat antrian pada BFS.

3. Deklarasi Class

```
public class GraphTraversal_2511531015 {
```

Bagian ini mendeklarasikan class bernama **GraphTraversal_2511531015**.

Class ini berisi program untuk:

1. Membuat graf.
2. Menambahkan edge.
3. Menampilkan graf.
4. Melakukan penelusuran DFS.
5. Melakukan penelusuran BFS.

4. Membuat Variabel Graph

```
private Map<String, List<String>> graph = new HashMap<>();
```

Kode ini digunakan untuk menyimpan graf dalam bentuk **Adjacency List**.

Penjelasannya:

```
Map<String, List<String>>
```

Artinya setiap node bertipe String, dan setiap node memiliki daftar tetangga berupa List<String>.

Contoh isi grafnya bisa seperti ini:

A -> B, C

B -> A, D, E

C -> A

D -> B

E -> B

HashMap digunakan untuk menyimpan pasangan data antara node dan daftar tetangganya.

5. Method addEdge

```
public void addEdge(String node1_1015, String node2_1015) {  
    graph.putIfAbsent(node1_1015, new ArrayList<>());  
    graph.putIfAbsent(node2_1015, new ArrayList<>());  
    graph.get(node1_1015).add(node2_1015);  
    graph.get(node2_1015).add(node1_1015);  
}
```

Method ini digunakan untuk menambahkan hubungan atau edge antara dua node.

Karena graf ini adalah **graf tidak berarah**, maka hubungan dibuat dua arah.

Misalnya:

```
graph.addEdge("A", "B");
```

Maka hasilnya:

A terhubung ke B

B terhubung ke A

Bagian ini:

```
graph.putIfAbsent(node1_1015, new ArrayList<>());  
graph.putIfAbsent(node2_1015, new ArrayList<>());
```

berfungsi untuk memastikan bahwa node sudah ada di dalam graf. Jika belum ada, maka node akan dibuat dengan daftar tetangga kosong.

Kemudian:

```
graph.get(node1_1015).add(node2_1015);  
graph.get(node2_1015).add(node1_1015);
```

berfungsi untuk menghubungkan kedua node secara dua arah.

6. Method printGraph

```
public void printGraph() {  
    System.out.println("Graf Awal (Adjacency List):");  
    for (String node : graph.keySet()) {  
        System.out.print(node + "->");  
        List<String> neighbors = graph.get(node);  
        System.out.println(String.join(" ", neighbors));  
    }  
    System.out.println();  
}
```

Method ini digunakan untuk menampilkan isi graf dalam bentuk **Adjacency List**.

Bagian ini:

```
for (String node : graph.keySet())
```

digunakan untuk mengambil semua node yang ada di dalam graf.

Bagian ini:

```
List<String> neighbors = graph.get(node);
```

digunakan untuk mengambil daftar tetangga dari node tersebut.

Kemudian:

```
System.out.println(String.join(" ", neighbors));
```

digunakan untuk mencetak semua tetangga node dengan pemisah koma.

Contoh output:

Graf Awal (Adjacency List):

A->B, C

B->A, D, E

C->A

D->B

E->B

7. Method dfs

```
public void dfs(String start) {  
    Set<String> visited = new HashSet<>();  
    System.out.println("Penelusuran DFS:");  
    dfsHelper(start, visited);  
    System.out.println();  
}
```

Method ini digunakan untuk menjalankan **DFS** atau **Depth First Search**.

DFS adalah metode penelusuran graf yang berjalan sedalam mungkin terlebih dahulu sebelum kembali ke node sebelumnya.

Bagian ini:

```
Set<String> visited = new HashSet<>();
```

digunakan untuk menyimpan node yang sudah dikunjungi agar tidak dikunjungi ulang.

Bagian ini:

```
dfsHelper(start, visited);
```

memanggil method bantuan dfsHelper() untuk melakukan proses DFS secara rekursif.

8. Method dfsHelper

```
private void dfsHelper(String current, Set<String> visited) {  
    if (visited.contains(current)) return;  
    visited.add(current);  
    System.out.print(current + " ");  
    for (String neighbor : graph.getDefault(current, new ArrayList<>())) {  
        dfsHelper(neighbor, visited);  
    }  
}
```

Method ini adalah inti dari proses DFS.

Bagian ini:

```
if (visited.contains(current)) return;
```

berfungsi untuk menghentikan proses jika node sudah pernah dikunjungi.

Bagian ini:

```
visited.add(current);
```

digunakan untuk menandai node saat ini sebagai node yang sudah dikunjungi.

Bagian ini:

```
System.out.print(current + " ");
```

digunakan untuk mencetak node yang sedang dikunjungi.

Kemudian:

```
for (String neighbor : graph.getDefault(current, new ArrayList<>())) {  
    dfsHelper(neighbor, visited);  
}
```

digunakan untuk mengunjungi semua tetangga dari node saat ini secara rekursif.

Jika grafnya seperti ini:

A terhubung ke B dan C

B terhubung ke D dan E

Maka DFS dari A bisa menghasilkan:

A B D E C

9. Method bfs

```
public void bfs(String start) {  
    Set<String> visited = new HashSet<>();  
    Queue<String> queue = new LinkedList<>();  
    queue.add(start);  
    visited.add(start);  
    System.out.println("Penelusuran BFS:");  
    while (!queue.isEmpty()) {  
        String current = queue.poll();  
        System.out.print(current + " ");  
        for (String neighbor : graph.getOrDefault(current, new ArrayList<>())) {  
            if (!visited.contains(neighbor)) {  
                queue.add(neighbor);  
                visited.add(neighbor);  
            }  
        }  
    }  
    System.out.println();  
}
```

Method ini digunakan untuk menjalankan **BFS** atau **Breadth First Search**.

BFS adalah metode penelusuran graf yang mengunjungi node secara melebar terlebih dahulu. Artinya, BFS akan mengunjungi semua tetangga dari sebuah node sebelum masuk lebih jauh ke level berikutnya.

Bagian ini:

```
Set<String> visited = new HashSet<>();
```

digunakan untuk menyimpan node yang sudah dikunjungi.

Bagian ini:

```
Queue<String> queue = new LinkedList<>();
```

digunakan untuk membuat antrian. BFS menggunakan prinsip **FIFO** atau **First In First Out**, yaitu data yang masuk lebih dulu akan diproses lebih dulu.

Bagian ini:

```
queue.add(start);  
visited.add(start);
```

berarti node awal dimasukkan ke antrian dan ditandai sebagai sudah dikunjungi.

Bagian ini:

```
while (!queue.isEmpty())
```

berarti proses BFS akan terus berjalan selama antrian masih berisi node.

Bagian ini:

```
String current = queue.poll();
```

digunakan untuk mengambil node pertama dari antrian.

Kemudian:

```
for (String neighbor : graph.getDefault(current, new ArrayList<>()))
```

digunakan untuk mengecek semua tetangga dari node saat ini.

Jika tetangga belum dikunjungi:

```
if (!visited.contains(neighbor)) {  
    queue.add(neighbor);  
    visited.add(neighbor);  
}
```

maka tetangga tersebut dimasukkan ke antrian dan ditandai sebagai sudah dikunjungi.

Jika grafnya seperti ini:

A terhubung ke B dan C

B terhubung ke D dan E

Maka BFS dari A bisa menghasilkan:

A B C D E

10. Method main

```
public static void main(String[] args) {
```

Method main() adalah method utama yang pertama kali dijalankan oleh Java.

Di dalam method ini, program membuat objek graf, menambahkan edge, mencetak graf, lalu menjalankan DFS dan BFS.

11. Membuat Objek Graph

```
GraphTraversal_2511531015 graph = new GraphTraversal_2511531015();
```

Kode ini digunakan untuk membuat objek graf baru bernama graph.

Objek ini nantinya digunakan untuk memanggil method seperti:

```
addEdge()  
printGraph()  
dfs()  
bfs()
```

12. Membuat Graf

```
graph.addEdge("A", "B");  
graph.addEdge("A", "C");  
graph.addEdge("B", "D");  
graph.addEdge("B", "E");
```

Kode ini digunakan untuk membuat hubungan antar node.

Hubungan yang dibuat adalah:

A terhubung dengan B

A terhubung dengan C

B terhubung dengan D

B terhubung dengan E

Karena grafnya tidak berarah, maka bentuk grafnya adalah:

```
A
  /\
B  C
  /\
D  E
```

Dalam bentuk adjacency list:

```
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B
```

13. Mencetak Graf

```
System.out.println("Graf Awal Adalah: ");
graph.printGraph();
```

Kode ini digunakan untuk menampilkan graf awal yang sudah dibuat.

Outputnya kira-kira:

Graf Awal Adalah:

Graf Awal (Adjacency List):

```
A->B, C
B->A, D, E
C->A
D->B
E->B
```

Urutan output bisa sedikit berbeda karena HashMap tidak selalu mempertahankan urutan data.

14. Melakukan Penelusuran DFS dan BFS

```
graph.dfs("A");  
graph.bfs("A");
```

Kode ini digunakan untuk menelusuri graf mulai dari node A.

DFS akan menelusuri graf dengan cara masuk sedalam mungkin terlebih dahulu.

Contoh hasil DFS:

Penelusuran DFS:

A B D E C

BFS akan menelusuri graf dengan cara melebar terlebih dahulu.

Contoh hasil BFS:

Penelusuran BFS:

A B C D E

FOTO KODE PROGRAM DAN OUTPUT

```

package pekar_2511531015;

public class Node_2511531015 {
    int data_1015;
    Node_2511531015 left_1015;
    Node_2511531015 right_1015;

    public Node_2511531015(int data_1015) {
        this.data_1015 = data_1015;
        left_1015 = null;
        right_1015 = null;
    }

    public void setleft_1015(Node_2511531015 node) {
        if (left_1015 == null) {
            left_1015 = node;
        }
    }

    public void setright_1015(Node_2511531015 node) {
        if (right_1015 == null) {
            right_1015 = node;
        }
    }

    public Node_2511531015 getleft_1015() {
        return left_1015;
    }

    public Node_2511531015 getright_1015() {
        return right_1015;
    }

    public int getData_1015() {
        return data_1015;
    }

    public void setData_1015(int data_1015) {
        this.data_1015 = data_1015;
    }

    void printPreorder(Node_2511531015 node) {
        if (node == null) {
            return;
        }
        System.out.print(node.data_1015 + " ");
        printPreorder(node.left_1015);
        printPreorder(node.right_1015);
    }

    void printPostorder(Node_2511531015 node) {
        if (node == null) {
            return;
        }
        printPostorder(node.left_1015);
        printPostorder(node.right_1015);
        System.out.print(node.data_1015 + " ");
    }

    void printInorder(Node_2511531015 node) {
        if (node == null) {
            return;
        }
        printInorder(node.left_1015);
        System.out.print(node.data_1015 + " ");
        printInorder(node.right_1015);
    }

    public String print() {
        return this.print("", true, "");
    }

    public String print(String prefix, boolean isTail, String sb) {
        if (right_1015 != null) {
            right_1015.print(prefix + (isTail ? " " : "  "), false, sb);
        }
        System.out.print(prefix + (isTail ? "(-/- " : "/- ") + data_1015);
        if (left_1015 != null) {
            left_1015.print(prefix + (isTail ? " " : "  "), true, sb);
        }
        return sb;
    }
}

```

```

package pekar_2511531015;

public class Node_2511531015 {
    int data_1015;
    Node_2511531015 left_1015;
    Node_2511531015 right_1015;

    public Node_2511531015(int data_1015) {
        this.data_1015 = data_1015;
        left_1015 = null;
        right_1015 = null;
    }

    public void setleft_1015(Node_2511531015 node) {
        if (left_1015 == null) {
            left_1015 = node;
        }
    }

    public void setright_1015(Node_2511531015 node) {
        if (right_1015 == null) {
            right_1015 = node;
        }
    }

    public Node_2511531015 getleft_1015() {
        return left_1015;
    }

    public Node_2511531015 getright_1015() {
        return right_1015;
    }

    public int getData_1015() {
        return data_1015;
    }

    public void setData_1015(int data_1015) {
        this.data_1015 = data_1015;
    }

    void printPreorder(Node_2511531015 node) {
        if (node == null) {
            return;
        }
        System.out.print(node.data_1015 + " ");
        printPreorder(node.left_1015);
        printPreorder(node.right_1015);
    }

    void printPostorder(Node_2511531015 node) {
        if (node == null) {
            return;
        }
        printPostorder(node.left_1015);
        printPostorder(node.right_1015);
        System.out.print(node.data_1015 + " ");
    }

    void printInorder(Node_2511531015 node) {
        if (node == null) {
            return;
        }
        printInorder(node.left_1015);
        System.out.print(node.data_1015 + " ");
        printInorder(node.right_1015);
    }

    public String print() {
        return this.print("", true, "");
    }

    public String print(String prefix, boolean isTail, String sb) {
        if (right_1015 != null) {
            right_1015.print(prefix + (isTail ? " " : "  "), false, sb);
        }
        System.out.print(prefix + (isTail ? "(-/- " : "/- ") + data_1015);
        if (left_1015 != null) {
            left_1015.print(prefix + (isTail ? " " : "  "), true, sb);
        }
        return sb;
    }
}

```

Graf Awal Adalah:
 Graf Awal (Adjacency List):
 A->B, C
 B->A, D, E
 C->A
 D->B
 E->B

Penelusuran DFS:
 A B D E C
 Penelusuran BFS:
 A B C D E

Gambar 1.1 Node_2511531015

```
1 package pekar_251151015;
2 public class BTree_251151015 {
3     private Node_251151015 root_3015;
4     private Node_251151015 currentNode;
5
6     public BTree_251151015() {
7         root_3015 = null;
8     }
9
10    public boolean search(int data_3015) {
11        return search(root_3015, data_3015);
12    }
13
14    private boolean search(Node_251151015 node_3015, int data_3015) {
15        if (node_3015.getData_3015() == data_3015)
16            return true;
17        if (node_3015.getLeft_3015() != null)
18            if (search(node_3015.getLeft_3015(), data_3015))
19                return true;
20        if (node_3015.getRight_3015() != null)
21            if (search(node_3015.getRight_3015(), data_3015))
22                return true;
23        return false;
24    }
25
26    public void printInorder() {
27        root_3015.printInorder(root_3015);
28    }
29
30    public void printPreorder() {
31        root_3015.printPreorder(root_3015);
32    }
33
34    public void printPostorder() {
35        root_3015.printPostorder(root_3015);
36    }
37
38    public Node_251151015 getroot() {
39        return root_3015;
40    }
41
42    public boolean isEmpty() {
43        return root_3015 == null;
44    }
45
46    public int countNode() {
47        return countNode(root_3015);
48    }
49
50    private int countNode(Node_251151015 node_3015) {
51        int count_3015 = 1;
52        if (node_3015 != null) {
53            count_3015 += countNode(node_3015.getLeft_3015());
54            count_3015 += countNode(node_3015.getRight_3015());
55            return count_3015;
56        }
57        return 0;
58    }
59
60    public void print() {
61        root_3015.print();
62    }
63
64    public Node_251151015 getCurrent() {
65        return currentNode;
66    }
67
68    public void setCurrent(Node_251151015 node_3015) {
69        this.currentNode = node_3015;
70    }
71
72    public void setRoot(Node_251151015 root_3015) {
73        this.root_3015 = root_3015;
74    }
75
76    }
77
78    }
79
80    }
81
82    }
83
84    }
85
86    }
87
88    }
89
90    }
91
92    }
93
94    }
95
96    }
97
98    }
99
100   }
```

```
1 package pekar_251151015;
2 public class BTree_251151015 {
3     private Node_251151015 root_3015;
4     private Node_251151015 currentNode;
5
6     public BTree_251151015() {
7         root_3015 = null;
8     }
9
10    public boolean search(int data_3015) {
11        return search(root_3015, data_3015);
12    }
13
14    private boolean search(Node_251151015 node_3015, int data_3015) {
15        if (node_3015.getData_3015() == data_3015)
16            return true;
17        if (node_3015.getLeft_3015() != null)
18            if (search(node_3015.getLeft_3015(), data_3015))
19                return true;
20        if (node_3015.getRight_3015() != null)
21            if (search(node_3015.getRight_3015(), data_3015))
22                return true;
23        return false;
24    }
25
26    public void printInorder() {
27        root_3015.printInorder(root_3015);
28    }
29
30    public void printPreorder() {
31        root_3015.printPreorder(root_3015);
32    }
33
34    public void printPostorder() {
35        root_3015.printPostorder(root_3015);
36    }
37
38    public Node_251151015 getroot() {
39        return root_3015;
40    }
41
42    public boolean isEmpty() {
43        return root_3015 == null;
44    }
45
46    public int countNode() {
47        return countNode(root_3015);
48    }
49
50    private int countNode(Node_251151015 node_3015) {
51        int count_3015 = 1;
52        if (node_3015 != null) {
53            count_3015 += countNode(node_3015.getLeft_3015());
54            count_3015 += countNode(node_3015.getRight_3015());
55            return count_3015;
56        }
57        return 0;
58    }
59
60    public void print() {
61        root_3015.print();
62    }
63
64    public Node_251151015 getCurrent() {
65        return currentNode;
66    }
67
68    public void setCurrent(Node_251151015 node_3015) {
69        this.currentNode = node_3015;
70    }
71
72    public void setRoot(Node_251151015 root_3015) {
73        this.root_3015 = root_3015;
74    }
75
76    }
77
78    }
79
80    }
81
82    }
83
84    }
85
86    }
87
88    }
89
90    }
91
92    }
93
94    }
95
96    }
97
98    }
99
100   }
```

```
1 package pekar_251151015;
2 public class BTree_251151015 {
3     private Node_251151015 root_3015;
4     private Node_251151015 currentNode;
5
6     public BTree_251151015() {
7         root_3015 = null;
8     }
9
10    public boolean search(int data_3015) {
11        return search(root_3015, data_3015);
12    }
13
14    private boolean search(Node_251151015 node_3015, int data_3015) {
15        if (node_3015.getData_3015() == data_3015)
16            return true;
17        if (node_3015.getLeft_3015() != null)
18            if (search(node_3015.getLeft_3015(), data_3015))
19                return true;
20        if (node_3015.getRight_3015() != null)
21            if (search(node_3015.getRight_3015(), data_3015))
22                return true;
23        return false;
24    }
25
26    public void printInorder() {
27        root_3015.printInorder(root_3015);
28    }
29
30    public void printPreorder() {
31        root_3015.printPreorder(root_3015);
32    }
33
34    public void printPostorder() {
35        root_3015.printPostorder(root_3015);
36    }
37
38    public Node_251151015 getroot() {
39        return root_3015;
40    }
41
42    public boolean isEmpty() {
43        return root_3015 == null;
44    }
45
46    public int countNode() {
47        return countNode(root_3015);
48    }
49
50    private int countNode(Node_251151015 node_3015) {
51        int count_3015 = 1;
52        if (node_3015 != null) {
53            count_3015 += countNode(node_3015.getLeft_3015());
54            count_3015 += countNode(node_3015.getRight_3015());
55            return count_3015;
56        }
57        return 0;
58    }
59
60    public void print() {
61        root_3015.print();
62    }
63
64    public Node_251151015 getCurrent() {
65        return currentNode;
66    }
67
68    public void setCurrent(Node_251151015 node_3015) {
69        this.currentNode = node_3015;
70    }
71
72    public void setRoot(Node_251151015 root_3015) {
73        this.root_3015 = root_3015;
74    }
75
76    }
77
78    }
79
80    }
81
82    }
83
84    }
85
86    }
87
88    }
89
90    }
91
92    }
93
94    }
95
96    }
97
98    }
99
100   }
```

```

Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder :
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|
|-- 7
|
|-- 3
|   |-- 6
|   |-- 9
|-- 1
    |-- 5
    |-- 2
        |-- 8
        |-- 4

```

Gambar 1.2 BTree_2511531015 & BTreeDriver_2511531015

```

// BTreeDriver.java
import java.util.*;

public class BTreeDriver_2511531015 {
    public static void main(String[] args) {
        BTree btree = new BTree(3);
        btree.add(1);
        btree.add(2);
        btree.add(4);
        btree.add(8);
        btree.add(5);
        btree.add(3);
        btree.add(6);
        btree.add(9);
        btree.add(7);

        System.out.println("InOrder traversal:");
        btree.inOrder();

        System.out.println("PreOrder traversal:");
        btree.preOrder();

        System.out.println("PostOrder traversal:");
        btree.postOrder();

        System.out.println("Displaying tree structure:");
        btree.display();
    }
}

```

```

// BTree.java
import java.util.*;

public class BTree_2511531015 {
    private Node root;
    private boolean[] visited;
    private Queue<String> queue;

    public BTree(int degree) {
        root = new Node();
        visited = new boolean[degree];
        queue = new LinkedList<>();
    }

    public void add(int value) {
        Node newNode = new Node(value);
        if (root == null) {
            root = newNode;
        } else {
            addInternal(root, newNode, 0);
        }
    }

    private void addInternal(Node root, Node newNode, int level) {
        if (level == root.degree) {
            root.children.add(newNode);
        } else {
            for (Node child : root.children) {
                if (!visited[child.value]) {
                    addInternal(child, newNode, level + 1);
                    visited[child.value] = true;
                }
            }
        }
    }

    public void inOrder() {
        inOrderInternal(root);
    }

    private void inOrderInternal(Node root) {
        if (root != null) {
            for (Node child : root.children) {
                inOrderInternal(child);
            }
            System.out.print(root.value + " ");
        }
    }

    public void preOrder() {
        preOrderInternal(root);
    }

    private void preOrderInternal(Node root) {
        if (root != null) {
            System.out.print(root.value + " ");
            for (Node child : root.children) {
                preOrderInternal(child);
            }
        }
    }

    public void postOrder() {
        postOrderInternal(root);
    }

    private void postOrderInternal(Node root) {
        if (root != null) {
            for (Node child : root.children) {
                postOrderInternal(child);
            }
            System.out.print(root.value + " ");
        }
    }

    public void display() {
        displayInternal(root, 0);
    }

    private void displayInternal(Node root, int level) {
        if (level == 0) {
            System.out.println("Root: " + root.value);
        } else {
            System.out.println("Level " + level + ":");
            for (Node child : root.children) {
                System.out.print(child.value + " ");
            }
            System.out.println();
        }
    }
}

```

```
Graf Awal Adalah:  
Graf Awal (Adjacency List):  
A->B, C  
B->A, D, E  
C->A  
D->B  
E->B
```

```
Penelusuran DFS:  
A B D E C  
Penelusuran BFS:  
A B C D E
```

Gambar 1.3 GraphTraversal_2511531015

KESIMPULAN

Berdasarkan materi dan kode program yang telah dibuat, dapat disimpulkan bahwa struktur data **Binary Tree** dan **Graph Traversal** merupakan materi penting dalam pemrograman, khususnya dalam memahami cara penyimpanan dan penelusuran data. Pada program **Node_2511531015**, setiap simpul dibuat untuk menyimpan data serta memiliki dua cabang, yaitu anak kiri dan anak kanan. Class ini menjadi dasar dalam pembentukan struktur pohon biner.

Pada program **BTree_2511531015**, dilakukan pengelolaan terhadap struktur Binary Tree, seperti menentukan root, menghitung jumlah simpul, mencari data, serta menampilkan isi pohon menggunakan metode traversal **InOrder**, **PreOrder**, dan **PostOrder**. Program ini menunjukkan bahwa data dalam pohon dapat ditelusuri dengan beberapa cara sesuai kebutuhan.

Pada program **BtreeDriver_2511531015**, seluruh proses pembuatan Binary Tree dijalankan. Program ini membuat beberapa node, menghubungkannya menjadi struktur pohon, menampilkan jumlah simpul, serta mencetak hasil traversal dan bentuk pohon. Dengan adanya class driver ini, program Binary Tree dapat dijalankan dan diuji hasilnya.

Selain itu, pada program **GraphTraversal_2511531015**, dipelajari cara merepresentasikan graf menggunakan **adjacency list** serta melakukan penelusuran menggunakan metode **DFS** dan **BFS**. DFS melakukan penelusuran secara mendalam terlebih dahulu, sedangkan BFS melakukan penelusuran secara melebar berdasarkan level atau jarak dari node awal.

Dengan demikian, melalui praktikum ini dapat dipahami bahwa Binary Tree digunakan untuk menyusun data dalam bentuk hierarki, sedangkan Graph Traversal digunakan untuk menelusuri hubungan antar simpul dalam graf. Implementasi menggunakan bahasa Java membantu memahami konsep struktur data secara lebih nyata melalui pembuatan class, object, method, serta penggunaan struktur data seperti Map, List, Set, dan Queue.